

The F# Path to Relaxation

Don Syme |> @dsymetweets
Principal Researcher, Microsoft

CodeMotion Italian Edition 2021

Hello

{code~~z~~motion}

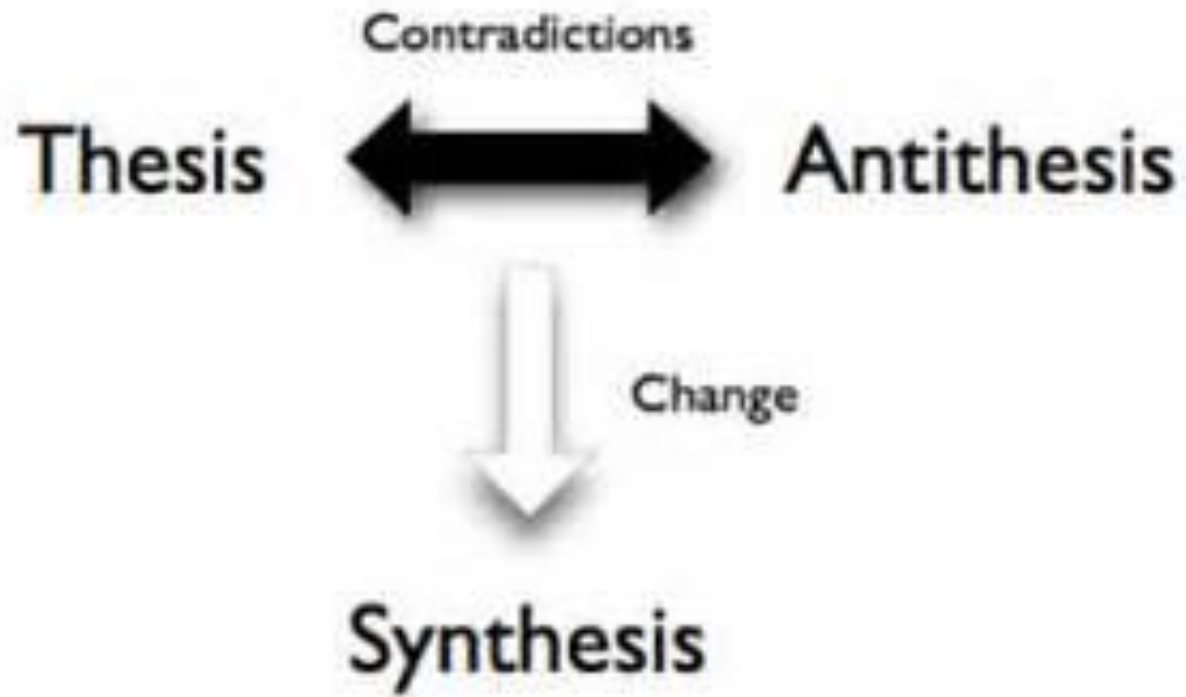
A Gentle Stroll Through the Land of Language Design and Delivery

A starting position....

Great disputes should be struggled with

They are chances to make a simpler, better, more
relaxed world

Not a new idea...



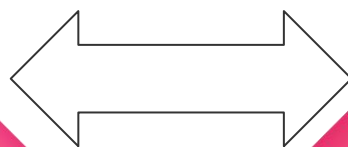
The Hegel

X

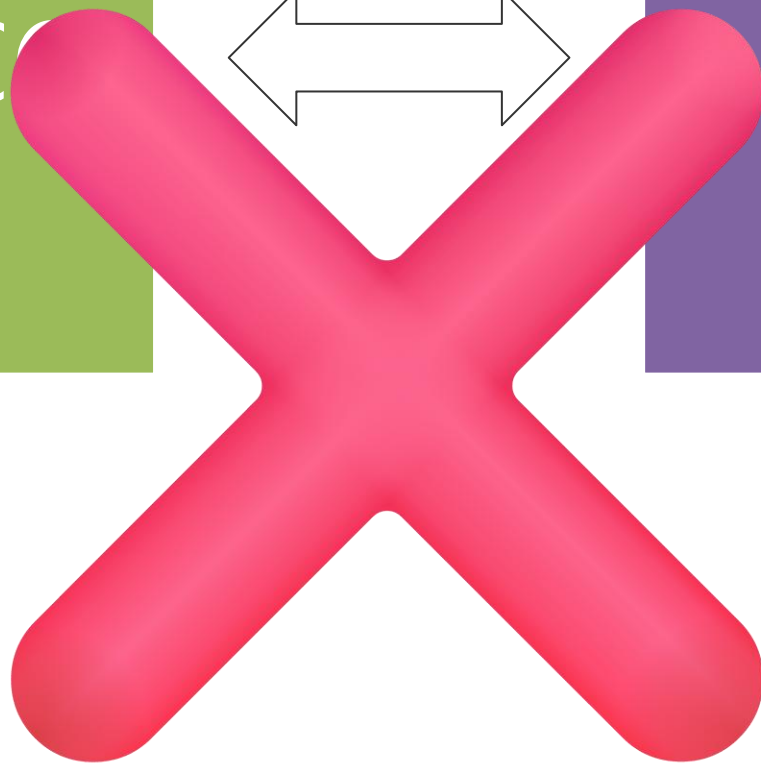


Y

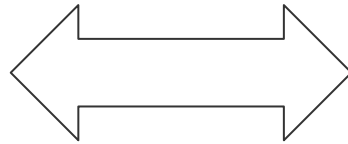
Whitespace



Tabs



Functional



Interop

Way back in time..... 2003

Functional languages were excellent but **isolated, non-interoperable, using their own VMs**. Interop standards like COM, CORBA, XML were a mess for language-integration.

The F# Approach (also Scala, Swift, ...)

Embrace industry-standard runtime layers, and influence them.

Design for end-to-end interop.

Compromise where needed.

For more!

The Early History of F# - HOPL IV (2021)

fsharp.org/history

Roll forward to today!

F# get started

- `dotnet new -lang F#`

`dotnet build`

F# tools are part of
the .NET SDK,
available everywhere



www.dot.net

F# for the web backend

- `dotnet new -i "giraffe-template::*"`

`dotnet giraffe`

High perf, functional
server-side
programming



A functional ASP.NET Core micro
web framework for building rich
web applications.

github.com/giraffe-fsharp/Giraffe

F# for the web frontend (JavaScript)

- `dotnet new -i "Fable.Template::*"`

```
dotnet new fable  
npm install  
npm start
```

You can use F# as a
JavaScript language



fable.io

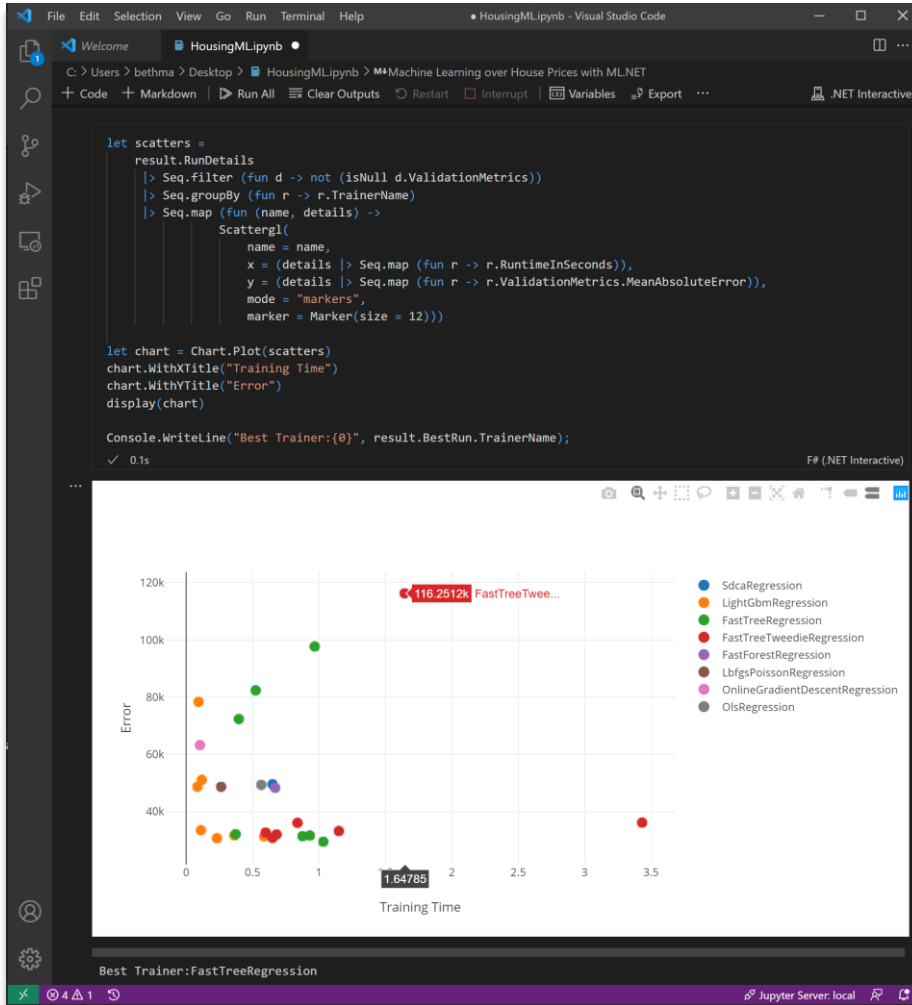
F# for the full stack with SAFE

- `dotnet new -i SAFE.Template`

```
dotnet new SAFE  
dotnet tool restore  
dotnet run
```



F# for data science & ML



github.com/dotnet/interactive

- `#r "nuget:Microsoft.ML,1.4.0"`
- `#r "nuget:Microsoft.ML.AutoML,0.16.0"`

The ML.NET website banner features the text 'ML.NET An open source and cross-platform machine learning framework' with buttons for 'Get started' and 'Model Builder'. Below the banner, four feature highlights are listed:

- Built for .NET developers**: With ML.NET, you can use your existing .NET skills to easily integrate ML into your .NET apps without any prior ML experience.
- Custom ML made easy with AutoML**: ML.NET offers AutoML and productive tools to help you easily build, train, and deploy high-quality custom ML models.
- Extended with TensorFlow & more**: ML.NET allows you to leverage other popular ML libraries like Infer.NET, TensorFlow, and ONNX for additional ML scenarios.
- Trusted and proven at scale**: Use the same ML framework used by recognized Microsoft products like PowerBI, Microsoft Defender, Outlook, and Bing.

dot.net/ml

What is F# good at?

dot.net/fsharp

F# is good for **programming**

dot.net/fsharp

F# is good for **web programming**

dot.net/fsharp

F# is good for **cloud programming**

dot.net/fsharp

F# is good for **AI/ML/data-science
programming**

dot.net/fsharp

F# is good for **succinct programming**

dot.net/fsharp

F# is good for **correct programming**

dot.net/fsharp

F# is good for **performant programming**

dot.net/fsharp

F# is good for **programming**

dot.net/fsharp

A functional-first approach makes a huge
difference in practice

fsharp.org/testimonials

An analysis (Simon Cousins, Energy Sector)

350,000

lines of C# OO
by offshore team

The C# project took five years and peaked at ~8 devs. It never fully implemented all of the contracts.

The F# project took less than a year and peaked at three devs (only one had prior experience with F#). All of the contracts were fully implemented.

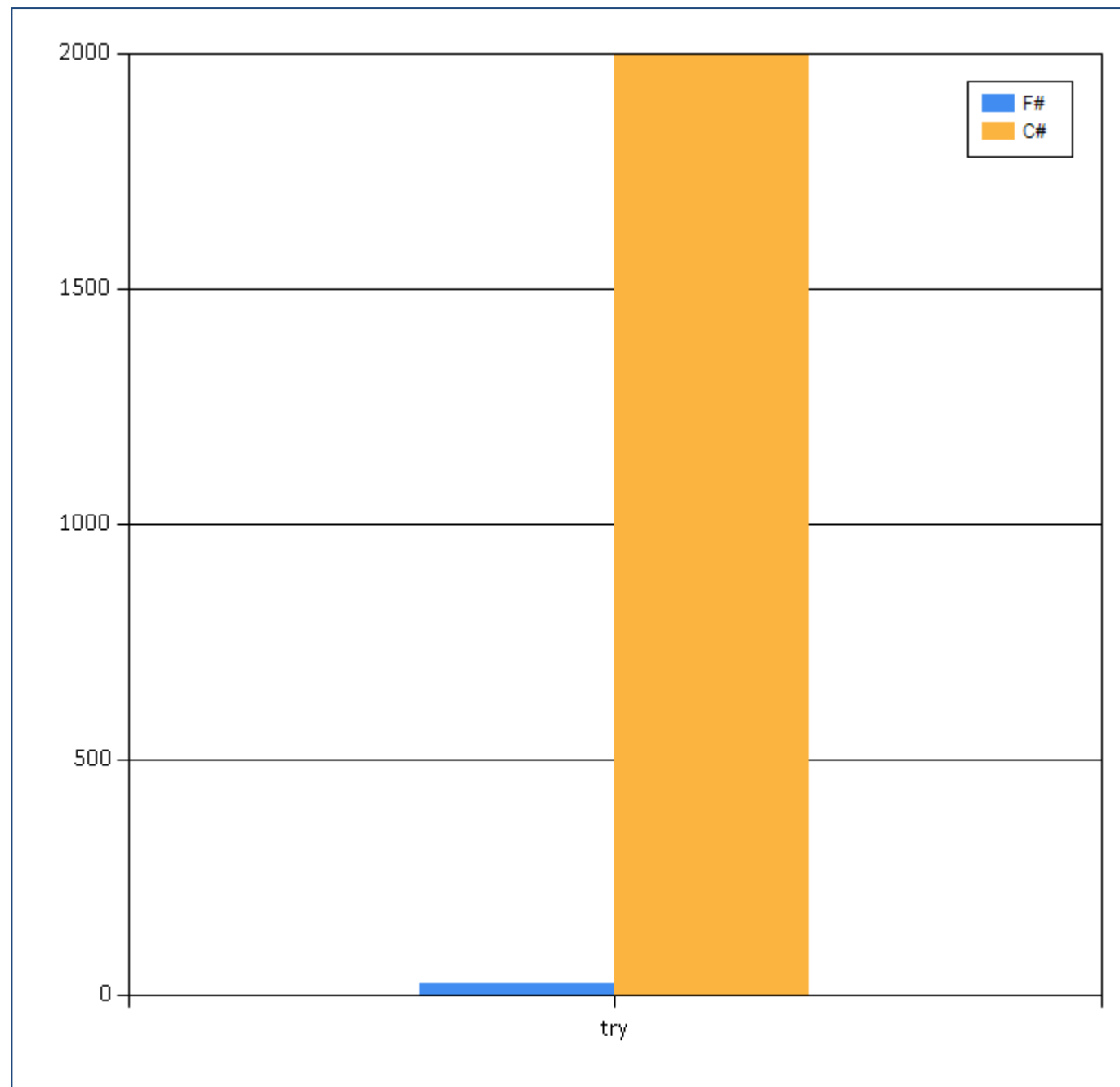
30,000

lines of robust F#, with
parallel + more features

An application to evaluate the revenue due from [Balancing Services](#) contracts in the UK energy industry

<http://simontcousins.azurewebsites.net/does-the-language-you-use-make-a-difference-revisited/>

Implementation	C#	F#
Braces	56,929	643
Blanks	29,080	3,630
Null Checks	3,011	15
Comments	53,270	487
Useful Code	163,276	16,667
App Code	305,566	21,442
Test Code	42,864	9,359
Total Code	348,430	30,801



Simon Cousins, Energy Sector

Zero

bugs in deployed system

“F# is the safe choice for this project,
any other choice is too risky”

An application to evaluate the revenue due from [Balancing Services](#) contracts in the UK energy industry

<http://simontcousins.azurewebsites.net/does-the-language-you-use-make-a-difference-revisited/>

A \$3B Unicorn, Built on F#

← → ↻ ⓘ <https://techcrunch.com/2016/08/07/walmart-buys-jet-com-for-3-billion/> 📌 ☆

TC News Startups Mobile Gadgets Trending 📈 Tesla Google Facebook 🔍

Advertising Tech jet.com Walmart Labs Walmart

Walmart is buying Jet.com for \$3 billion

Posted Aug 7, 2016 by [Jonathan Shieber \(@jshieber\)](#), [John Mannes \(@JohnMannes\)](#)

💬 f t in g+ 📺 📌 📧 📧 Next Story ➔



Walmart Stores is buying Jet.com in a deal worth \$3 billion dollars according to a source with direct knowledge of the deal, confirming reports that have been pouring in about the bid for Jet.com all week.

According to our source the signatures for the deal were dry on Friday and will be announced as early as Monday morning — echoing what was reported in both Bloomberg and Recode.

← → ↻ ⓘ www.managedsolution.com/jet-built-its-entire-e-commerce-platform-including-development-and-delivery-infrastructure-on-microsoft-azure/ ☆ ⋮

 MANAGED SOLUTION 🔍

Home About + Products & Services + Client Support + Contact Us Try Before You Buy!

Jet built its entire e-commerce platform, including development and delivery infrastructure, on Microsoft #Azure.

Jet.com – E-commerce challenger eyes the top spot, runs on the Microsoft cloud

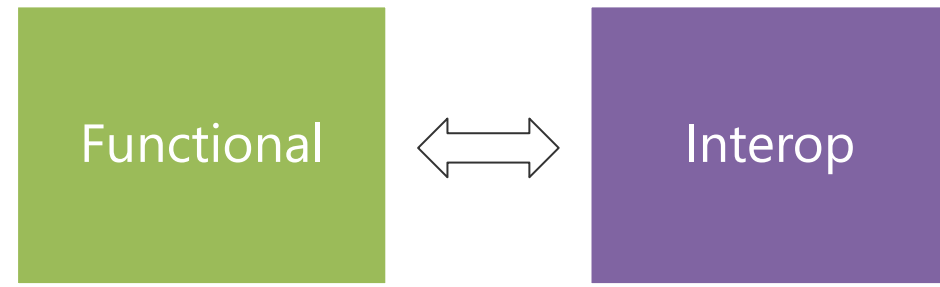
Marc Lore is perhaps best known as the creator of the popular e-commerce site Diapers.com, which was eventually sold to Amazon. Now, the entrepreneur and his team are ready to compete head-on with the e-retailing giant through an innovative online marketplace called Jet.com. To get up and running quickly, Jet built its entire e-commerce platform, including development and delivery infrastructure, on Microsoft Azure, using both .NET and open-source technologies.

Business Challenge

In 2010, Marc Lore sold his company Quidsi (which ran e-retailing sites like Diapers.com and Soap.com) to Amazon for \$550 million. Four years later, Marc is competing against Amazon directly—with the creation of a new online marketplace called Jet.com.

There are many reasons to think that Lore might just pull it off. For one, he plans to eliminate any margins from product sales. The company's only source of revenue will come from membership dues, eliminating the kind of mark-ups that Amazon charges and passing the savings on to the customer. In addition, an innovative pricing engine will work to reduce or eliminate costs in the e-commerce value chain, especially fulfillment costs and marketplace commissions.

"Our pricing engine will continually work out the most cost-effective way to fulfill an order from merchant locations closest to the consumer," explains Lore, Co-Founder and CEO of Jet. "The engine will also figure out which merchants can fulfill most cheaply by putting multiple



Relaxation Achieved, 200% ✓

✓ Today's FP languages are interoperable

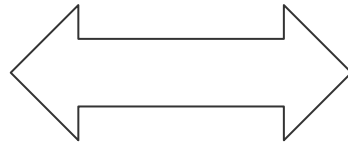
✓ Today's FP languages are practical

X



Y

Enterprise



Openness

TL;DR – Where we are now

F# is open, cross-
platform, neutral,
independent

All F# language and
tooling is accepting
contributions

The F# community is
very “self-empowered”
fsharp.org

Visual Studio for
Windows, Mac

Visual Studio Code for
Everywhere

.NET Interactive
Notebooks for
Everywhere

The F# Compiler
Service at the core

Fable for F#-to-
Javascript
SAFE for F# fullstack

F# 6.0 now complete!

The F# Language Design Process

github.com/fsharp/fslang-design

github.com/fsharp/fslang-suggestions

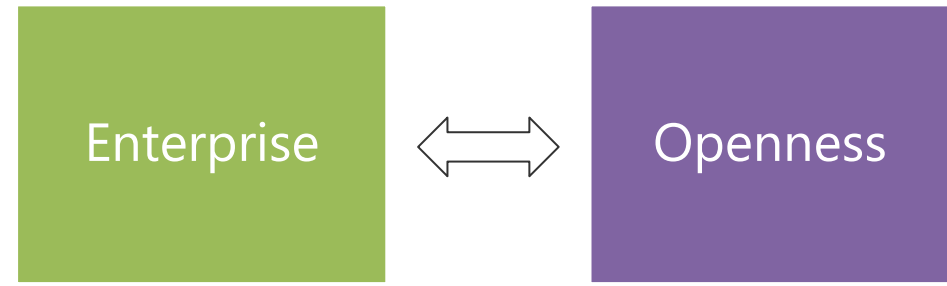
F# 5.0! (2021)

- ✓ #r nuget packages in scripts "#r \"nuget: Newtonsoft.Json\""
- ✓ Jupyter and .NET Interactive notebooks!
- ✓ string interpolation
- ✓ nameof
- ✓ applicatives syntax in computation expressions
- ✓ improved .NET interop
- ✓ improved Map/Set performance + more

<https://github.com/fsharp/fslang-design/tree/master/FSharp-5.0>

F# 6.0! (2021)

- ✓ `task { .. }`
- ✓ `expr[idx]`
- ✓ many language simplifications
- ✓ performance improvements
- ✓ much more!



Relaxation Achieved ✓

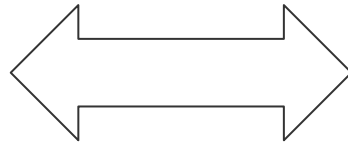
Enterprise Quality + Openness +
Community + Tooling +
Ecosystem + Evolution Path
=
Goodness

X



Y

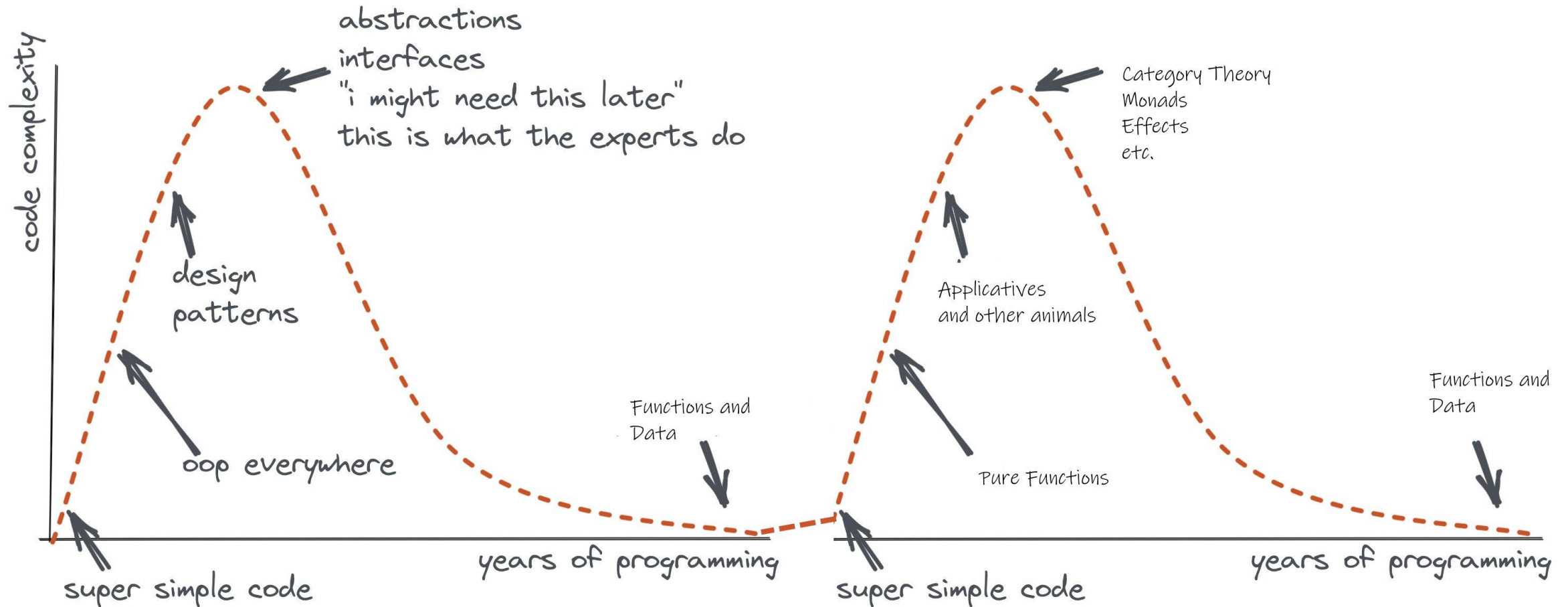
Functional



Objects

Remember the eternal sweet spot of programming!

Functions + Data (+ Some Types)



Then (2003)

In the 1990s functional languages were historically anti-OO

“Modern Compiler Implementation with Standard ML” - Chapter 14, Object Oriented Programming

object – to express a dislike or distaste for something

F# : Objects + Functional =

```
type Vector2D (dx:double, dy:double) =
```

```
    let d2 = dx*dx+dy*dy
```

```
    member v.DX = dx
```

```
    member v.DY = dy
```

```
    member v.Length = sqrt d2
```

```
    member v.Scale(k) = Vector2D (dx*k, dy*k)
```

Inputs to object construction

Object internals

Exported properties

Exported method

Objects

Constructed Class Types

```
type objectType(args) =  
  let internalValue = expr  
  let internalFunction args = expr  
  let mutable internalState = expr  
  member x.Prop1 = expr  
  member x.Meth2 args = expr
```

Object Interface Types

```
type IObject =  
  interface ISimpleObject  
    abstract Prop1 : type  
    abstract Meth2 : type -> type
```

Object Expressions

```
{ new IObject with  
  member x.Prop1 = expr  
  member x.Meth1 args = expr }  
  
{ new Object() with  
  member x.Prop1 = expr  
  interface IObject with  
    member x.Meth1 args = expr  
  interface IWidget with  
    member x.Meth1 args = expr }
```

Object Programming focuses on ...

succinct coding, notational convenience

API ergonomics

good naming

practical encapsulation

sensible, small, composable abstractions

expression-oriented

making simple things out of (potentially complex) foundations

Object-Oriented Programming too often means...

objects as a single paradigm

overuse of hierarchical classification

(Animal, Cat, Dog, AbstractJellyBeanFactoryDelegator)

large abstractions with many holes and failure points

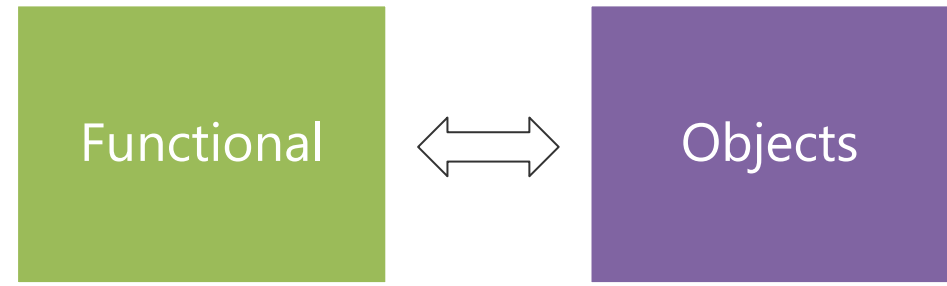
declarations not expressions

complex things by inheriting from complex things...

The F# approach

embrace objects ✓

avoid object-orientation ✗



Positive Synthesis Achieved ✓

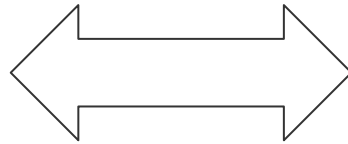
some limitations remain... and there is one area in particular that's worth discussing....

X



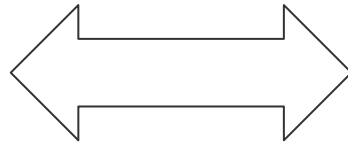
Y

Code



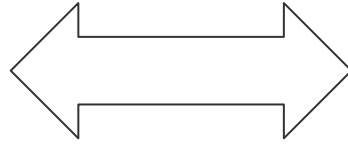
Data

Async



Simplicity

Pattern
Matching



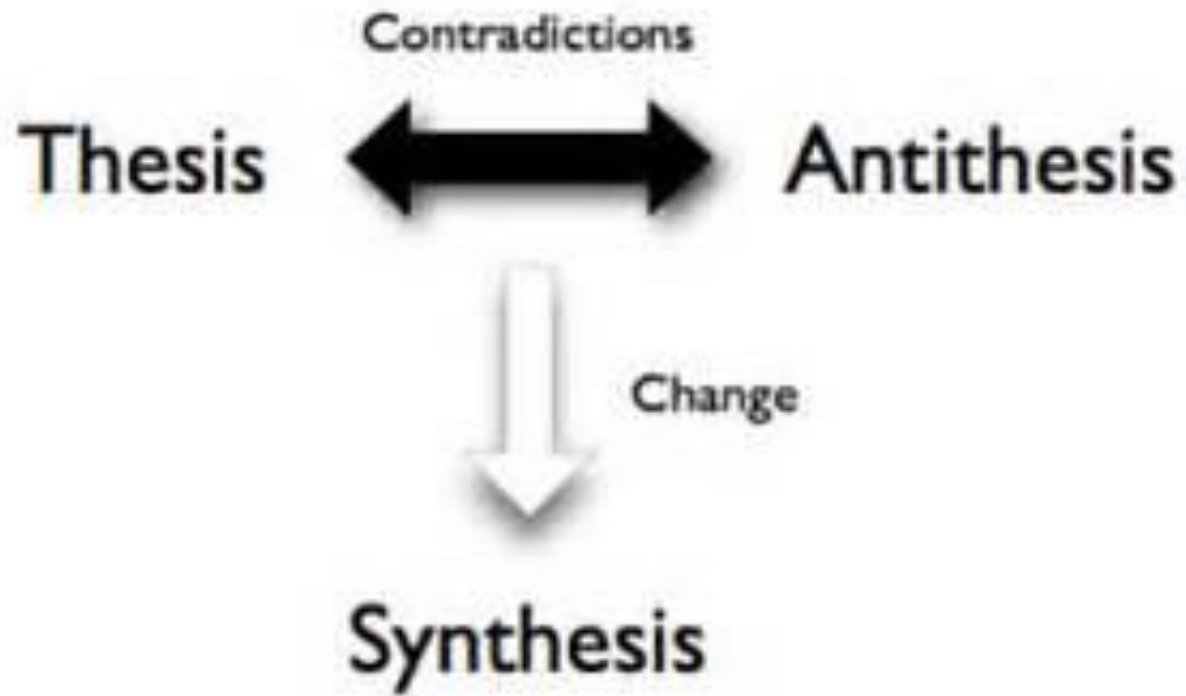
Abstraction

In Conclusion

F# is a magical language

It makes programming simple, logical and enjoyable

Remember the Hegel!



Resources

fsharp.org

dotnet.microsoft.com/learn/fsharp

[Don Teaches Guido F#](#)

Thank you!!

Questions?

X



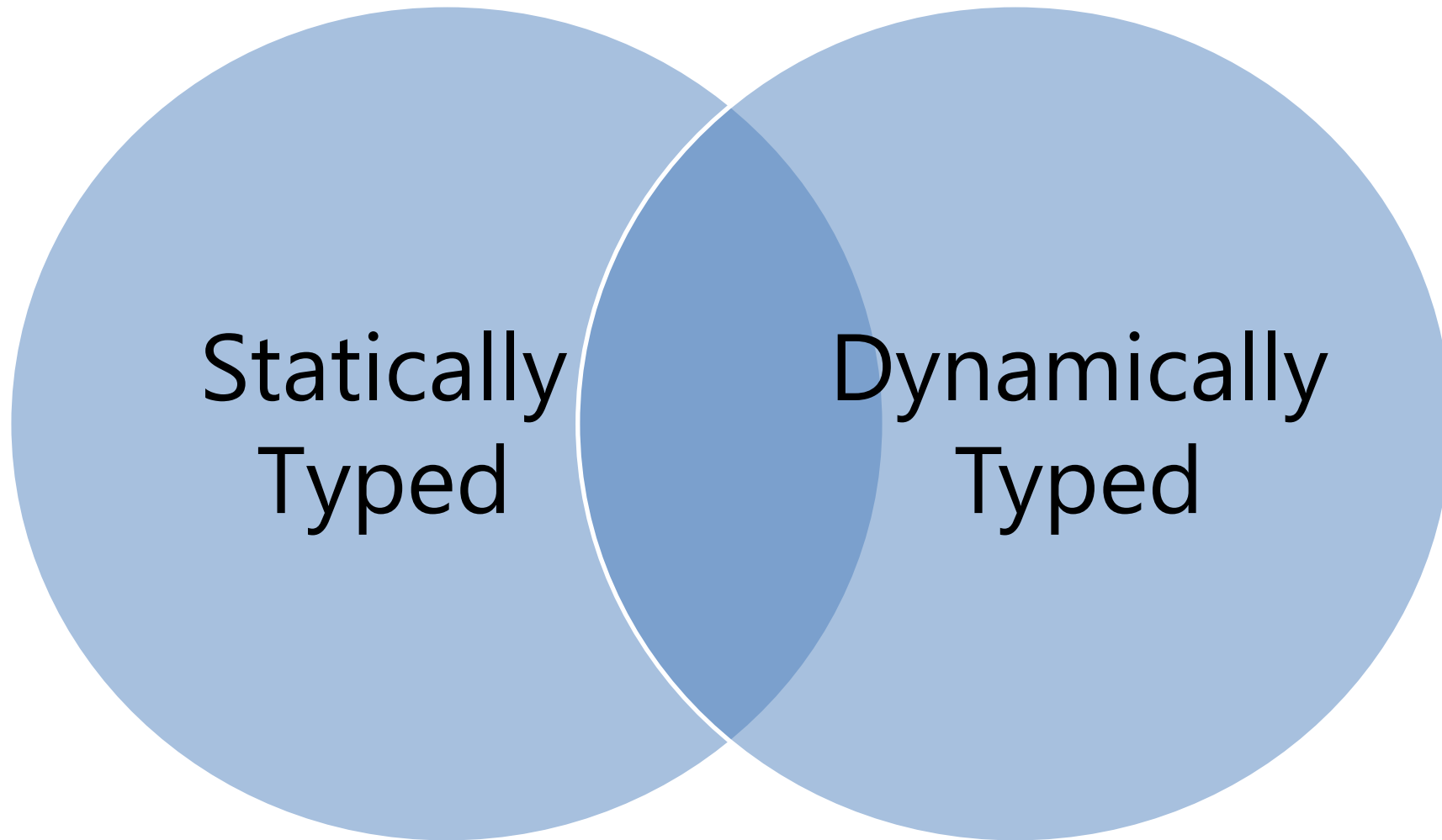
Y

We are living through an
information revolution

We need to bring information **into** the
language...

At internet-scale, strongly tooled, strongly typed

Paradigm Locator



Enter F# Type Providers....

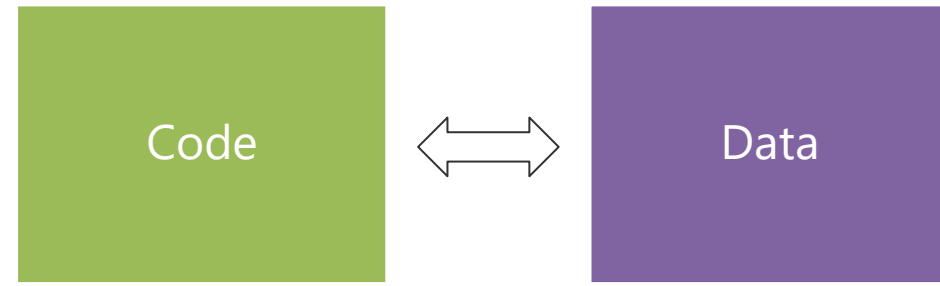
"Just like a library"

"A design-time component that computes a space of types and methods on-demand..."

"An adaptor between data/services and the .NET type system..."

"On-demand, scalable compile-time provision of type/module definitions..."

<https://fsprojects.github.io/FSharp.Data/images/json.gif>



Relaxation Achieved ✓ ✓

F# Type Providers have been a hugely successful feature – their range of useful application is enormous. They bring data and typed programming into synthesis like no other feature around

Help achieve relaxation today

Where there is conflict, find a positive, creative path
forward

Freedom and relaxation are possible!

The F# Compiler Service

fsharp.github.io/FSharp.Compiler.Service